

— TECHNICAL OVERVIEW

Interview Tracker

A recruiter's applicant-tracking workspace — candidate pipeline, résumé parsing with side-by-side validation, shared analytics, and one-click scheduling. Functionality and stack, as built and deployed.

● Live in production

▲ Vercel · interview-tracker-three.vercel.app

git · add-rag-explorer @ 7751d1e

FRONTEND

React 18

Vite 6 · JSX

BACKEND

Serverless

Vercel Functions

DATABASE

MongoDB

Atlas · driver 7.5

HOSTING

Vercel

JAMstack SPA

Authentication

Client-side demo auth — no server session.

- ◆ Email / password register & login
- ◆ SHA-256 hashing via Web Crypto
- ◆ Optional Google Sign-In (OAuth JWT)
- ◆ Accounts & session in localStorage

Web Crypto

OAuth

demo-only

Candidate Management

Config-driven CRUD over 30+ fields.

- ◆ Add / edit via grouped form
- ◆ Auto-generated Cand ID / Req ID
- ◆ Inline status change, instant persist
- ◆ Soft-delete with reason + Undo

FIELD_GROUPS

recoverable

Résumé Preview & Validate new

Upload a résumé, auto-fill the form, then verify the parse against the source document — side by side on one screen.

- ◆ Accepts PDF · DOCX · TXT · MD (10 MB cap, validated)
- ◆ Heuristic extraction: name, email, phone, location, education, experience, notice period
- ◆ Split view: form on the left, preview pane on the right
- ◆ Parsed-vs-Form comparison table — ✓ match / ✗ edited, updates live
- ◆ DOCX rendered as a formatted Word document; PDF in-frame; re-apply parsed values

pdfjs-dist

docx-preview

JSZip

regex heuristics

Bulk Import

Load an existing Submission Log.

- ◆ Excel (.xlsx/.xls), CSV, PDF
- ◆ Multi-sheet merge by Cand ID + Name
- ◆ Tolerant header mapping
- ◆ PDF table extraction by visual rows

SheetJS

pdfjs-dist

Search & Filter

Find any candidate by any detail.

- ◆ Full-text search across every field
- ◆ Date-Sourced calendar filter
- ◆ Status dropdown filter
- ◆ Dashboard drill-down by predicate

useMemo

predicate filters

Dashboard Analytics

Pipeline health at a glance.

- ◆ Clickable stat tiles → filtered list
- ◆ Breakdown bars by status / client
- ◆ Color-coded status pills

derived stats

Candidate Table

Operate the roster row by row.

- ◆ Date-aware sortable columns
- ◆ Inline status pills
- ◆ Per-row edit / delete
- ◆ Email invite + .ics calendar file

iCalendar

mailto

Scheduling & Sharing

Get the pipeline out of the app.

- ◆ Per-candidate .ics + email notify
- ◆ Excel export of the full log
- ◆ Email pipeline summary
- ◆ Microsoft Teams share deep-link

SheetJS

Teams

VCALENDAR

Persistence & Sync

One shared roster, resilient writes.

- ◆ Shared MongoDB store — identical counts for all
- ◆ localStorage cache for instant paint
- ◆ Debounced writes + dirty-guard vs. clobber
- ◆ sendBeacon flush on unload; deletion audit trail

MongoDB

sendBeacon

offline-first

02 TECHNOLOGY STACK

LAYER	TECHNOLOGY	DETAIL
Framework	React 18.3	Hooks only (state / memo / effect / ref); single-page tab state, no router or Redux
Build	Vite 6	plugin-react, ESM, dynamic imports code-split pdfjs & docx-preview into lazy chunks
Language	JavaScript (JSX)	No TypeScript
Styling	Plain CSS	CSS variables (theme tokens), glass / gradient UI, Plus Jakarta Sans
Backend	Vercel Functions	api/candidates.js & api/audit.js — Node serverless handlers
Database	MongoDB Atlas	mongodb 7.5; single-doc roster (atomic upsert) + deletions collection; pooled conn
Hosting	Vercel	vercel.json framework=vite, SPA rewrite to index.html; production deploy
PDF	pdfjs-dist 4.7	Text extraction for résumé parse & bulk PDF import; web worker
DOCX read	JSZip 3.10	Unzip word/document.xml, strip tags for parse heuristics
DOCX render	docx-preview	Formatted Word rendering in the preview pane (lazy-loaded)
Excel	xlsx (SheetJS) 0.18	Multi-sheet import (read) and export (write)
Auth crypto	Web Crypto API	SHA-256 password hashing (crypto.subtle)
Calendar	iCalendar	Hand-built VCALENDAR / VEVENT .ics generation
Integrations	mailto · Teams · Google	Email, Teams share deep-link, Google OAuth sign-in

JAMstack — React SPA + serverless API + MongoDB

Full-list-replace persistence. The client holds the whole roster and sends it on change; the server upserts a single document (`_id: "candidates"`) so writes stay atomic.

Offline-first. `localStorage` paints instantly and survives network loss; MongoDB is the source of truth. A dirty-flag + can-write guard prevents a slow initial fetch from clobbering unsaved edits, and `sendBeacon` flushes pending writes on unload.

Data hygiene. Legacy records are migrated on load — status / recruiter remap, date coercion to ISO, and name splitting into first / last.